Lab_Assignment_11.2.

HT NO:2303A51360

BATCH NO:29

TASK-1: Stack Using AI Guidance

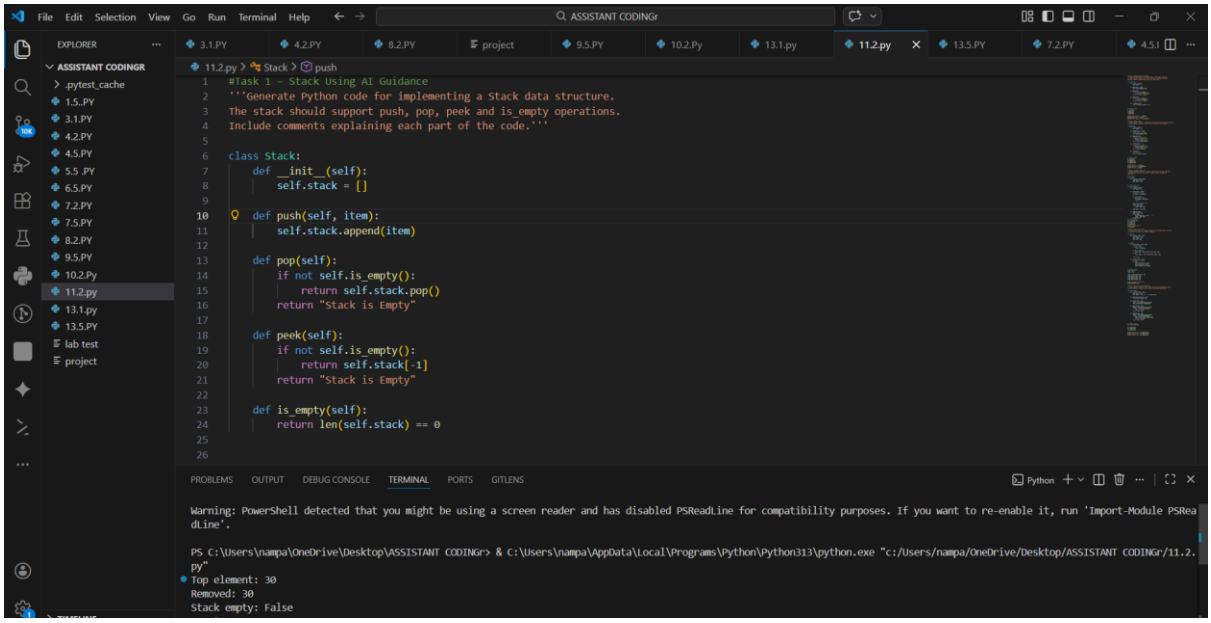
PROMPT:

Generate Python code for implementing a Stack data structure.

The stack should support push, pop, peek and is_empty operations.

Include comments explaining each part of the code.

CODE:



OBSERVATION:

The stack follows the **LIFO (Last In First Out)** principle.

The **push() operation** successfully adds elements to the top of the stack.

The **pop() operation** removes the most recently added element.

The **peek() function** correctly displays the top element without removing it.

The **is_empty() function** helps in checking whether the stack contains elements or not.

TASK-2 :

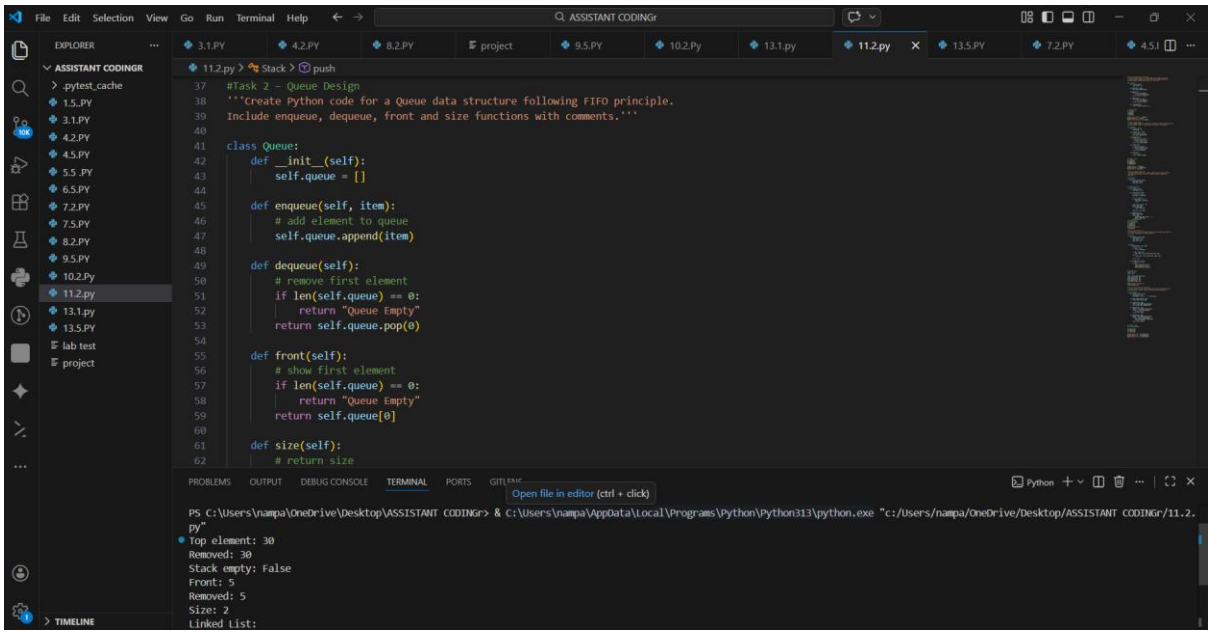
Queue Design

PROMPT:

Create Python code for a Queue data structure following FIFO principle.

Include enqueue, dequeue, front and size functions with comments.

CODE:



```
#Task 2 – Queue Design
'''Create Python code for a Queue data structure following FIFO principle.
Include enqueue, dequeue, front and size functions with comments.'''

class Queue:
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        # add element to queue
        self.queue.append(item)

    def dequeue(self):
        # remove first element
        if len(self.queue) == 0:
            return "Queue Empty"
        return self.queue.pop(0)

    def front(self):
        # show first element
        if len(self.queue) == 0:
            return "Queue Empty"
        return self.queue[0]

    def size(self):
        # return size
```

```
PS C:\Users\nampa\OneDrive\Desktop\ASSISTANT CODINGr> & C:\Users\nampa\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/nampa/OneDrive/Desktop/ASSISTANT CODINGr/11.2.py"
Top element: 30
Removed: 30
Stack empty: False
Front: 5
Removed: 5
Size: 2
Linked List:
```

OBSERVATION

The queue follows the **FIFO (First In First Out)** principle.

The **enqueue() operation** inserts elements at the rear of the queue.

The **dequeue() operation** removes the element from the front of the queue.

The **front() function** correctly displays the first element in the queue.

The **size() function** accurately returns the number of elements present.

TASK-3:

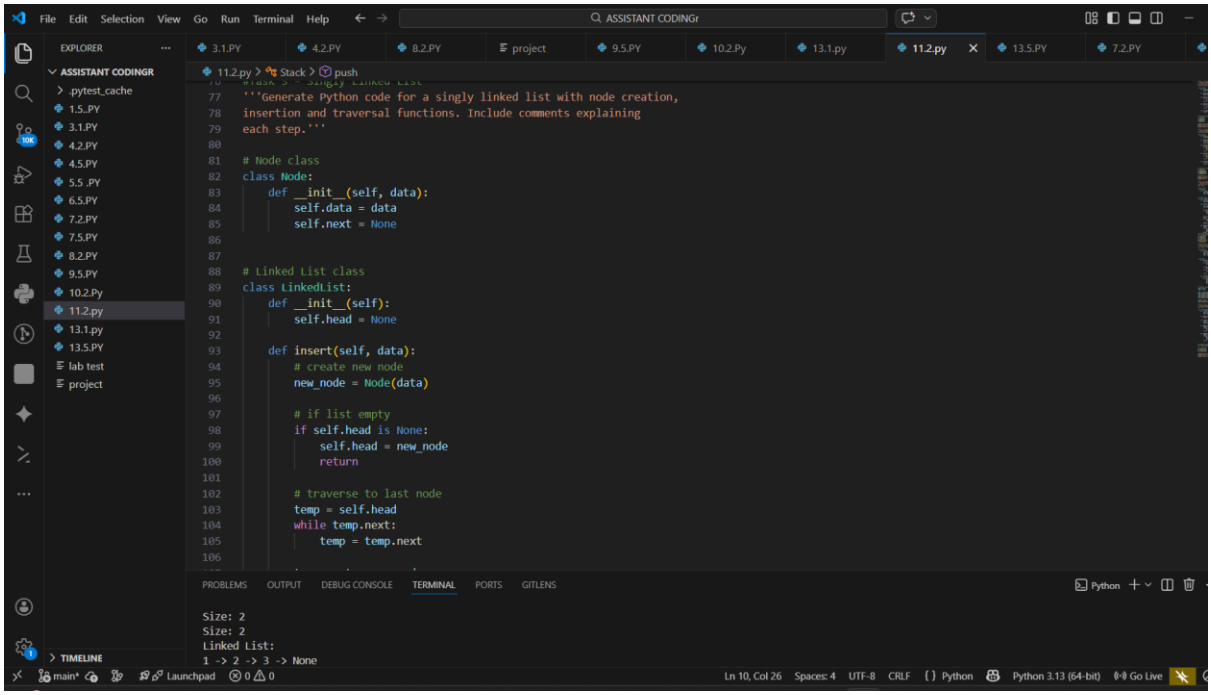
Singly Linked List

PROMPT:

Generate Python code for a singly linked list with node creation,

insertion and traversal functions. Include comments explaining each step.

CODE:



OBSERVATION:

The linked list stores elements in **nodes connected through pointers**.

Each node contains **data and a reference to the next node**.

The **insertion operation** correctly adds nodes at the end of the list.

The **display/traversal function** successfully visits and prints each node.

Linked lists provide **dynamic memory allocation**, unlike arrays.

TASK-4:

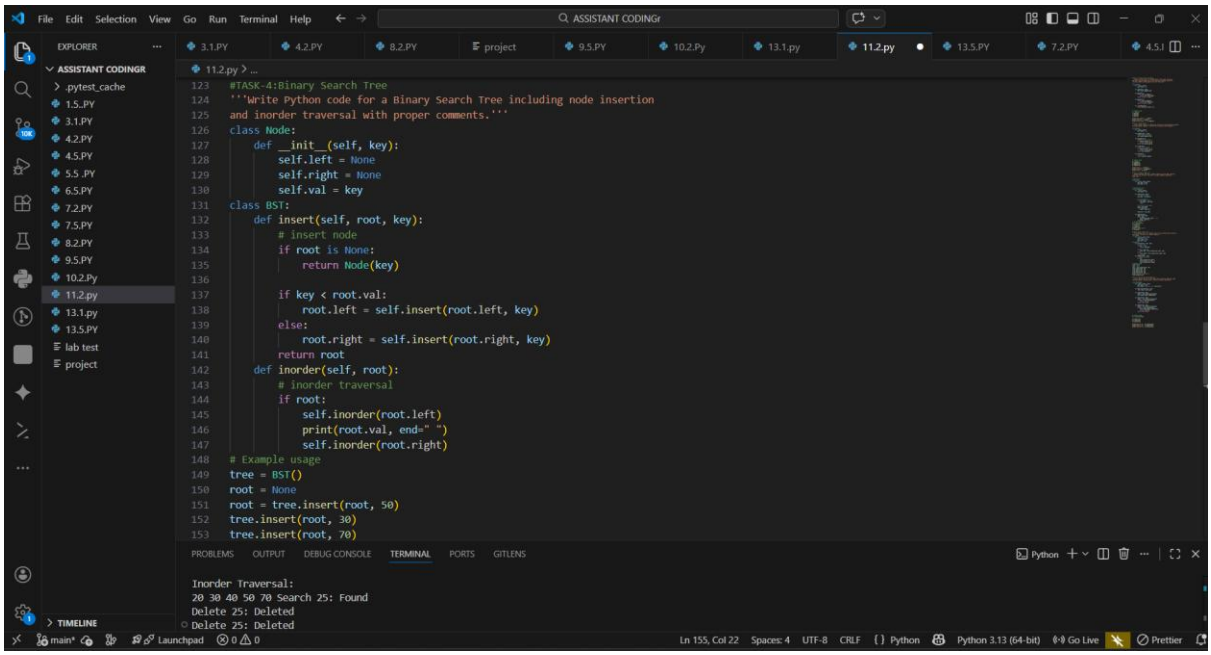
Binary Search Tree

PROMPT:

Write Python code for a Binary Search Tree including node insertion

and inorder traversal with proper comments.

CODE:



```python
#TASK-4:Binary Search Tree
'''Write Python code for a Binary Search Tree including node insertion
and inorder traversal with proper comments.'''
class Node:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
class BST:
    def insert(self, root, key):
        # insert node
        if root is None:
            return Node(key)

        if key < root.val:
            root.left = self.insert(root.left, key)
        else:
            root.right = self.insert(root.right, key)
        return root
    def inorder(self, root):
        # inorder traversal
        if root:
            self.inorder(root.left)
            print(root.val, end=" ")
            self.inorder(root.right)
# Example usage
tree = BST()
root = None
root = tree.insert(root, 50)
tree.insert(root, 30)
tree.insert(root, 70)
```

```
Inorder Traversal:
20 30 40 50 70 Search 25: Found
Delete 25: Deleted
Delete 25: Deleted
```

OBSERVATION:

The Binary Search Tree maintains the property **Left Node < Root < Right Node**.

The **insertion operation** correctly places nodes based on their values.

The **in-order traversal** prints the elements in **sorted order**.

BST improves search efficiency compared to linear structures.

The structure helps in implementing efficient **search, insertion, and deletion operations**.

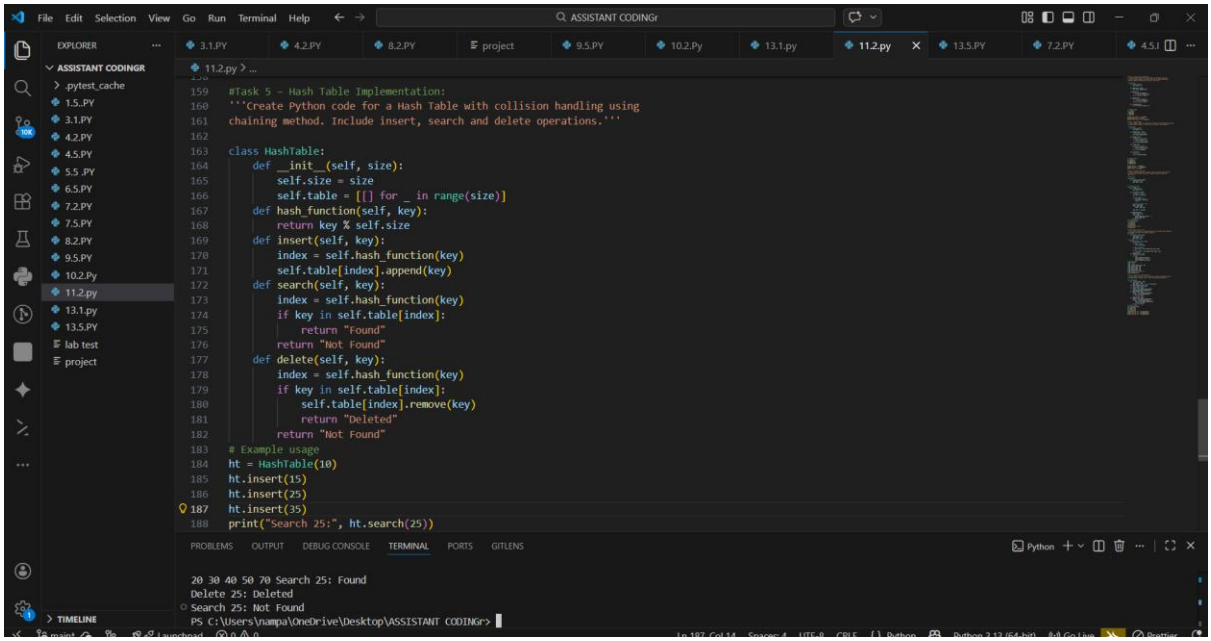
TASK-5:

Hash Table Implementation:

PROMPT:

Create Python code for a Hash Table with collision handling using chaining method. Include insert, search and delete operations

CODE:



```python
#Task 5 - Hash Table Implementation:
'''Create Python code for a Hash Table with collision handling using
chaining method. Include insert, search and delete operations.'''

class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(size)]
    def hash_function(self, key):
        return key % self.size
    def insert(self, key):
        index = self.hash_function(key)
        self.table[index].append(key)
    def search(self, key):
        index = self.hash_function(key)
        if key in self.table[index]:
            return "Found"
        return "Not Found"
    def delete(self, key):
        index = self.hash_function(key)
        if key in self.table[index]:
            self.table[index].remove(key)
            return "Deleted"
        return "Not Found"
# Example usage
ht = HashTable(10)
ht.insert(15)
ht.insert(25)
ht.insert(35)
print("Search 25:", ht.search(25))
```

```
20 30 40 50 70 Search 25: Found
Delete 25: Deleted
Search 25: Not Found
PS C:\Users\nampa\OneDrive\Desktop\ASSISTANT CODINGr>
```

OBSERVATION:

The hash table stores data using a **hash function to compute an index**.

The **insert operation** places elements into the calculated bucket.

The **search function** quickly locates elements using hashing.

Collision handling using chaining allows multiple elements at the same index.

The **delete function** successfully removes elements from the hash tabl